Universidad de los Andes
Departamento de Ingeniería de Sistemas y Computación
ISIS 1311 – Tecnología e Infraestructura de Cómputo

Caso 3 - 20261

Concurrencia y Sincronización de Procesos

El propósito de este caso es aplicar los conceptos de concurrencia y sincronización de procesos vistos en clase, a través del diseño e implementación de un simulador concurrente de un sistema IoT (Internet of Things) para un campus universitario que coordina la generación de eventos desde múltiples sensores (productores), realiza detección de anomalías (consumidores intermedios) y encamina la información a diferentes servicios.

1. Objetivos:

- Modelar interacciones concurrentes entre múltiples entidades (sensores, bróker de eventos, clasificadores) que cooperan y compiten por recursos compartidos.
- Implementar mecanismos de sincronización utilizando exclusivamente las primitivas básicas de Java (synchronized, wait, notify, notifyAll, yield, join, CyclicBarrier).
- Usar diferentes patrones de espera: espera pasiva y espera semi-activa.
- Analizar situaciones típicas de concurrencia, como condiciones de carrera y bloqueos mutuos, proponiendo soluciones correctas mediante sincronización.

MUY IMPORTANTE: El entregable de este caso debe ser 100% de su autoría. No está permitido usar ayudas no autorizadas (incluyendo chatbots o tecnologías similares). El incumplimiento de lo anterior resultará en una calificación de cero (0.0) para el caso.

Además, tenga en cuenta que los casos están diseñados para apoyar su proceso de aprendizaje e identificar y resolver sus dudas sobre los temas estudiados antes del parcial.

Finalmente, recuerde que en el parcial habrá una pregunta sobre el caso.

2. El Caso: Sistema IoT

Queremos simular un sistema IoT que administra eventos, los consolida y los despliega para un campus universitario. El sistema debe coordinar la recepción, filtrado y entrega de los datos, garantizando control de anomalías.

A. Arquitectura general del sistema

El sistema está organizado como una cadena de procesamiento de eventos en la que participan varios tipos de actores: Sensores, Bróker y Analizador, Administrador, Clasificadores y servidores, quienes siguen el comportamiento que la Figura 1 ilustra.

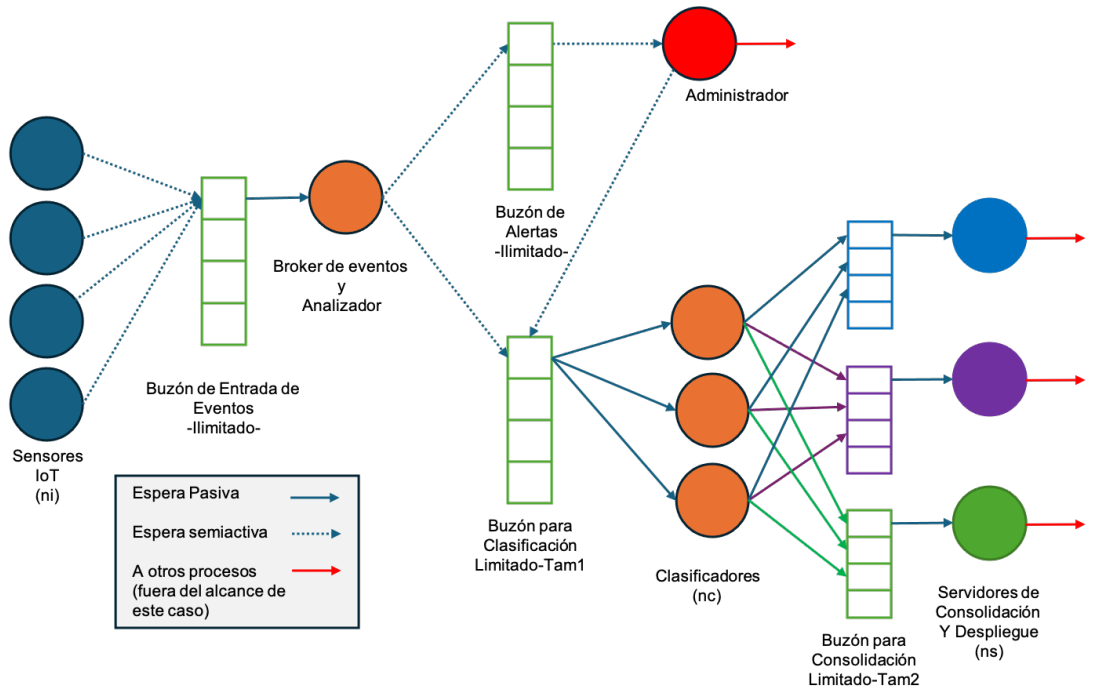

Figura 1. Arquitectura del Sistema de Mensajería

Descripción de los actores:

1. Sensores IoT
    - Son los responsables de generar los eventos.
    - Cada sensor produce un número asignado de eventos.
    - Los sensores depositan sus eventos en un buzón de entrada de eventos.
    - Cada sensor termina cuando ha generado todos los eventos asignados.
2. Broker y Analizador
    - Es el encargado de revisar los eventos del buzón de entrada.
    - Si el evento es sospechoso, lo envía al buzón de alertas.
    - Si es válido, lo envía al buzón para clasificación.
    - El bróker termina cuando ha procesado todos los eventos que el sistema debe generar.
3. Clasificadores
    - Leen los eventos del buzón de clasificación y dependiendo del tipo de evento lo envían al servidor de consolidación y despliegue correspondiente.
    - Un clasificador termina cuando recibe un evento de fin.

4. Administrador
    - Lee eventos del buzón de alertas.
    - Inspecciona los eventos en profundidad para determinar si son efectivamente maliciosos. Si confirma que un evento es malicioso lo descarta. Los eventos que pasan la inspección se llevan al buzón de clasificación.
    - El administrador termina cuando recibe un evento de fin.
5. Servidores de despliegue
    - Lee eventos de su propio buzón de consolidación.
    - Consolida y despliega los mensajes para los clientes.
    - Un servidor termina cuando recibe un mensaje de fin.

B. Funcionamiento del Sistema

El sistema funciona de la siguiente manera:

1. Sensores

- Cada Sensor (Thread productor) se inicia con un número fijo de eventos que debe generar. Este número se calcula con un valor base en el archivo de configuración multiplicado por el número de identificación del sensor (usaremos identificadores secuenciales, empezando en 1).
- Cada sensor genera sus eventos con un identificador único. El identificador debe generarse con el identificador del sensor y un secuencial.
- Cada evento debe incluir, adicionalmente, un número seudoaleatorio entre 1 y ns, donde ns es el número de servidores de consolidación y despliegue. Este valor indica dos cosas: el tipo del evento y el servidor al que el evento debe llegar.
- Los sensores depositan sus eventos en un buzón con capacidad ilimitada.
- Cada sensor termina su ejecución cuando termina de producir el número asignado de eventos.

2. Bróker de eventos

- Lee los eventos uno por uno y determina si son normales o representan alguna anomalía en el comportamiento esperado.
- En este caso vamos a simular la presencia de eventos anómalos con base en un número generado seudoaleatoriamente. Por cada evento revisado, el bróker genera un número entre 0 y 200, si el número es múltiplo de 8, el evento es considerado anómalo y enviado al buzón de alertas, si no el evento es considerado normal y enviado al buzón para clasificación.

- Cuando todos los eventos esperados han sido generados por los sensores, el bróker envía un evento de fin al administrador. Esto significa que el bróker debe conocer el número total de eventos esperados.

3. Administrador

- Lee los eventos sospechosos del buzón de alertas y determina si el comportamiento anómalo es efectivamente un problema o solo un evento fuera de lo común pero inofensivo.
- Vamos a simular la presencia de eventos sospechosos pero inofensivos por medio de un número seudoaleatorio. Por cada evento analizado el administrador genera un número entre 0 y 20. Si el número es múltiplo de 4, el evento es considerado normal y enviado al buzón para clasificación, si no el evento es descartado. Tenga en cuenta que en este caso estamos descartando las alertas confirmadas, pero en un contexto real, el administrador y la organización deben manejar las alertas confirmadas de forma apropiada.
- Cuando recibe el evento de fin termina. Pero antes de terminar, debe avisar a los clasificadores que deben terminar, para esto debe crear y depositar nc eventos de fin en el buzón para clasificación (uno por clasificador).

4. Clasificadores

- Cada clasificador retira eventos del buzón para clasificación, uno por uno, identifica el tipo de evento, y envía el evento al buzón del servidor de consolidación correspondiente.
- Cada clasificador termina cuando recibe un evento de fin.
- El último clasificador en terminar debe generar y enviar el evento de fin para los servidores de consolidación y despliegue, para esto debe crear y depositar ns eventos de fin; un evento en el buzón de cada servidor de consolidación.
- La condición previa significa que los clasificadores deben saber cuántos están corriendo de forma concurrente y registrar su terminación de alguna forma.

5. Servidores de consolidación y despliegue

- Cada servidor debe procesar los eventos normales que va tomando de su buzón de consolidación.
- La lectura de eventos se hace uno por uno. Para simular la lectura y el procesamiento, al recibir un evento lo procesan durante un tiempo aleatorio (entre 100 ms y 1000 ms) y luego quedan a la espera del siguiente evento.
- Termina su ejecución cuando recibe un evento de fin.

C. Terminación del sistema

- Los sensores terminan cuando han generado los eventos asignados.
- El bróker termina cuando ha procesado todos los eventos esperados en el sistema.
- El administrador, los clasificadores y los servidores de consolidación deben terminar cuando reciben un evento de fin (y cumplen con todas las condiciones descritas en las secciones previas).
- Todos los buzones deben quedar vacíos.

D. Parámetros configurables

El programa recibe un archivo texto que define los siguientes parámetros:

- Número de sensores (ni).
- Número base de eventos (un solo valor).
- Número de clasificadores (nc).
- Número de servidores de despliegue y consolidación (ns).
- Capacidad del buzón del buzón para clasificación (tam1).
- Capacidad de cada buzón para consolidación (tam2).

3. Informe

- En un archivo .zip entregue el código fuente del programa, y un informe explicando el diseño (diagrama de clase) y funcionamiento del programa.
- Incluya una sección de validación donde describa qué pruebas hizo para verificar los resultados.
- En el informe explique, para cada pareja de objetos que interactúan, cómo se realiza la sincronización.
- El nombre del archivo zip debe ser: caso3_login1_login2.zip

4. Condiciones de entrega

- El trabajo se debe realizar en grupos de 2 estudiantes. No debe haber consultas entre grupos. Recuerde que <u>debe conformar un grupo diferente a los casos anteriores</u> y debe trabajar en grupo.
- El grupo responde solidariamente por el contenido de todo el trabajo, y lo elabora conjuntamente (no es trabajo en grupo repartirse puntos o trabajos diferentes).
- Tenga en cuenta las reglas establecidas sobre la obligatoriedad del trabajo en grupo. Si no cumple con las reglas habrá una penalización de 0,5/5.
- En el parcial se incluirá una pregunta sobre el caso.

- El proyecto debe ser entregado por Bloque Neón por uno solo de los integrantes del grupo. **Al comienzo del informe, deben estar los nombres y carnés de los integrantes del grupo.** Si un integrante no aparece en el documento entregado, el grupo podrá informarlo posteriormente, sin embargo, habrá una penalización: la calificación asignada será distribuida (dividida de forma equitativa) entre los integrantes del grupo.
- **Se debe entregar por Bloque Neón <u>a más tardar</u> el 20 de abril a las 11:59 p.m.**
- **La fecha de entrega no será movida.**
- **Política de entrega tarde.** Para las entregas tarde, se aplicará la siguiente política: por cada 30 minutos o fracción de retraso, con respecto a la hora de entrega establecida en Bloque Neón, habrá una penalización de 0,5/5.
- En Bloque Neón, **es necesario inscribirse** en alguno de los grupos disponibles para poder acceder a la actividad de entrega del caso. Para ello, debe ingresar a la sección Grupos → Ver grupos disponibles → Caso 3. Es muy importante asegurarse de que ambos integrantes estén registrados en el mismo grupo; de lo contrario, quien no se inscriba no recibirá la información de la calificación.